Jupyter Notebook Execution Report

Name: Amanda Quintino dos Santos

Project Title: BU OMDS

Date: June 28, 2026

Cell 1: ■ Markdown

Week 2 Homework

Cell 2: ■ Code

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Lasso
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
```

Cell 3: ■ Code

```
predict_conversion_df = pd.read_csv("digital_marketing_campaign_dataset.csv")
google_ads_df = pd.read_csv("GoogleAds_DataAnalytics_Sales_Uncleaned.csv")
marketing_products_df = pd.read_csv("marketing_and_product_performance.csv")
```

Cell 4: ■ Markdown

Lasso Regression for predict_conversion_df

Cell 5: ■ Markdown

Lasso regression was chosen for predict_conversion_df because there is virtually no multicollinearity per the analysis performed in week 1. Lasso will force the coefficients of non-predictive variables to zero, providing a model that only isolates variables that actively drive conversions. Ridge and Elastic Net are better suited for situations involving highly correlated features.

Cell 6: ■ Code

```
# Identify categorical columns to encode
categorical_cols = predict_conversion_df.select_dtypes(include=['object',
'category']).columns.tolist()
```

```
# Drop non-predictive/target identifiers
X_all = predict_conversion_df.drop(columns=['CustomerID', 'Conversion',
'ConversionRate'], errors='ignore')
y_target = predict_conversion_df['Conversion']
```

```
# One-hot encode the categorical features
X_encoded = pd.get_dummies(X_all, columns=categorical_cols, drop_first=True)
```

```
# Train-test split
X_train_enc, X_test_enc, y_train_enc, y_test_enc = train_test_split(X_encoded,
y_target, test_size=0.2, random_state=42)
```

```
# Scale features for Lasso
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_enc)
X_test_scaled = scaler.transform(X_test_enc)
```

```
# Train Lasso Regression model
lasso = Lasso(alpha=0.1, random_state=42)
lasso.fit(X_train_scaled, y_train_enc)
```

```
# Predict and evaluate
y_pred = lasso.predict(X_test_scaled)
mse = mean_squared_error(y_test_enc, y_pred)
r2 = r2_score(y_test_enc, y_pred)
```

```
print(f"Lasso Regression Results:")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"R^2 Score: {r2:.4f}")
```

Output:

```
Lasso Regression Results:
Mean Squared Error (MSE): 0.1066
R^2 Score: -0.0001
```

Cell 7: ■ Markdown

Elastic Net and Ridge for google_ads_df

Cell 8: ■ Markdown

Both Elastic Net and Ridge regression were used for google_ads_df to compare them. Per week 1's analysis, the baseline features had low VIF scores but in creating polynomial and interaction terms we introduce multicollinearity. Instead of dropping features arbitrarily, ridge will handle the multicollinearity by shrinking correlated features. Elastic net on the other hand provides a middle ground with the feature selection of Lasso as it maintains the stability of Ridge.

Cell 9: ■ Code

```
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import Ridge
```

```
# Clean data: drop rows with missing target values
google_ads_df_clean = google_ads_df.dropna(subset=['Conversions']).copy()
```

```
# Drop non-predictive/target identifiers and select numeric
X_ga =
google_ads_df_clean.select_dtypes(include=[np.number]).drop(columns=['Conversions',
'Conversion Rate'], errors='ignore')
```

```
# Impute missing values with mean for features
X_ga = X_ga.fillna(X_ga.mean())
```

```
y_ga = google_ads_df_clean['Conversions']
```

```
# Train-test split (80% / 20%)
X_train_ga, X_test_ga, y_train_ga, y_test_ga = train_test_split(X_ga, y_ga,
test_size=0.2, random_state=42)
```

```
# Generate Polynomial Features (degree=2)
poly_ga = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly_ga = poly_ga.fit_transform(X_train_ga)
X_test_poly_ga = poly_ga.transform(X_test_ga)
```

```
# Regularization models like Ridge require features to be scaled
scaler_ga = StandardScaler()
X_train_scaled_ga = scaler_ga.fit_transform(X_train_poly_ga)
```

```
X_test_scaled_ga = scaler_ga.transform(X_test_poly_ga)
```

```
# Train the Ridge Regression model
ridge_ga = Ridge(alpha=1.0, random_state=42)
ridge_ga.fit(X_train_scaled_ga, y_train_ga)
```

```
# Make predictions and evaluate
y_pred_ridge_ga = ridge_ga.predict(X_test_scaled_ga)
mse_ridge_ga = mean_squared_error(y_test_ga, y_pred_ridge_ga)
r2_ridge_ga = r2_score(y_test_ga, y_pred_ridge_ga)
```

```
print(f"Ridge Regression Results:")
print(f"Mean Squared Error (MSE): {mse_ridge_ga:.4f}")
print(f"R^2 Score: {r2_ridge_ga:.4f}")
```

Output:

```
Ridge Regression Results:
Mean Squared Error (MSE): 5.0823
R^2 Score: 0.0010

[STDERR]
c:\Users\santo\OneDrive\Documents\GitHub\Learning\.venv\Lib\site-packages\pandas\core\generic.py:
You are setting values through chained assignment. Currently this works in certain cases, but whe
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/ind

  result[k] = res_k
```

Cell 10: ■ Code

```
from sklearn.linear_model import ElasticNet
```

```
# Train the Elastic Net Regression model
elastic_ga = ElasticNet(alpha=0.1, l1_ratio=0.5, random_state=42)
elastic_ga.fit(X_train_scaled_ga, y_train_ga)
```

```
# Make predictions and evaluate
```

```
y_pred_elastic_ga = elastic_ga.predict(X_test_scaled_ga)
mse_elastic_ga = mean_squared_error(y_test_ga, y_pred_elastic_ga)
r2_elastic_ga = r2_score(y_test_ga, y_pred_elastic_ga)
```

```
print(f"Elastic Net Regression Results:")
print(f"Mean Squared Error (MSE): {mse_elastic_ga:.4f}")
print(f"R^2 Score: {r2_elastic_ga:.4f}")
```

Output:

```
Elastic Net Regression Results:
Mean Squared Error (MSE): 5.0870
R^2 Score: 0.0001
```

Cell 11: ■ Markdown

Elastic Net for marketing_products_df

Cell 12: ■ Markdown

Elastic net is being used for marketing_products_df due to its high multicollinearity. Elastic net uses an L1 penalty to diminish unimportant interactions while it levarges L2 penalty to properly distribute feature weights.

Cell 13: ■ Code

```
# Select numeric features and drop non-predictive/target identifiers
numeric_cols_mp = marketing_products_df.select_dtypes(include=[np.number]).columns
features_to_drop_mp = ['Campaign_ID', 'Product_ID', 'Customer_ID', 'Flash_Sale_ID', 'Bundle_ID', 'Conversions', 'Revenue_Generated', 'ROI']
X_mp = marketing_products_df[numeric_cols_mp].drop(columns=features_to_drop_mp, errors='ignore')
y_mp = marketing_products_df['Conversions']
```

```
# Train-test split (80% / 20%)
X_train_mp, X_test_mp, y_train_mp, y_test_mp = train_test_split(X_mp, y_mp, test_size=0.2, random_state=42)
```

```
# Generate Polynomial Features (degree=2)
poly_mp = PolynomialFeatures(degree=2, include_bias=False)
```

```
X_train_poly_mp = poly_mp.fit_transform(X_train_mp)
X_test_poly_mp = poly_mp.transform(X_test_mp)
```

```
# Scale features
scaler_mp = StandardScaler()
X_train_scaled_mp = scaler_mp.fit_transform(X_train_poly_mp)
X_test_scaled_mp = scaler_mp.transform(X_test_poly_mp)
```

```
# Train the Elastic Net Regression model
elastic_mp = ElasticNet(alpha=0.1, l1_ratio=0.5, random_state=42)
elastic_mp.fit(X_train_scaled_mp, y_train_mp)
```

```
# Predict and evaluate
y_pred_elastic_mp = elastic_mp.predict(X_test_scaled_mp)
mse_elastic_mp = mean_squared_error(y_test_mp, y_pred_elastic_mp)
r2_elastic_mp = r2_score(y_test_mp, y_pred_elastic_mp)
```

```
print(f"Elastic Net Regression Results:")
print(f"Mean Squared Error (MSE): {mse_elastic_mp:.4f}")
print(f"R^2 Score: {r2_elastic_mp:.4f}")
```

Output:

```
Elastic Net Regression Results:
Mean Squared Error (MSE): 84838.0068
R^2 Score: -0.0021
```